The Significance of Test Driven Development (TDD)

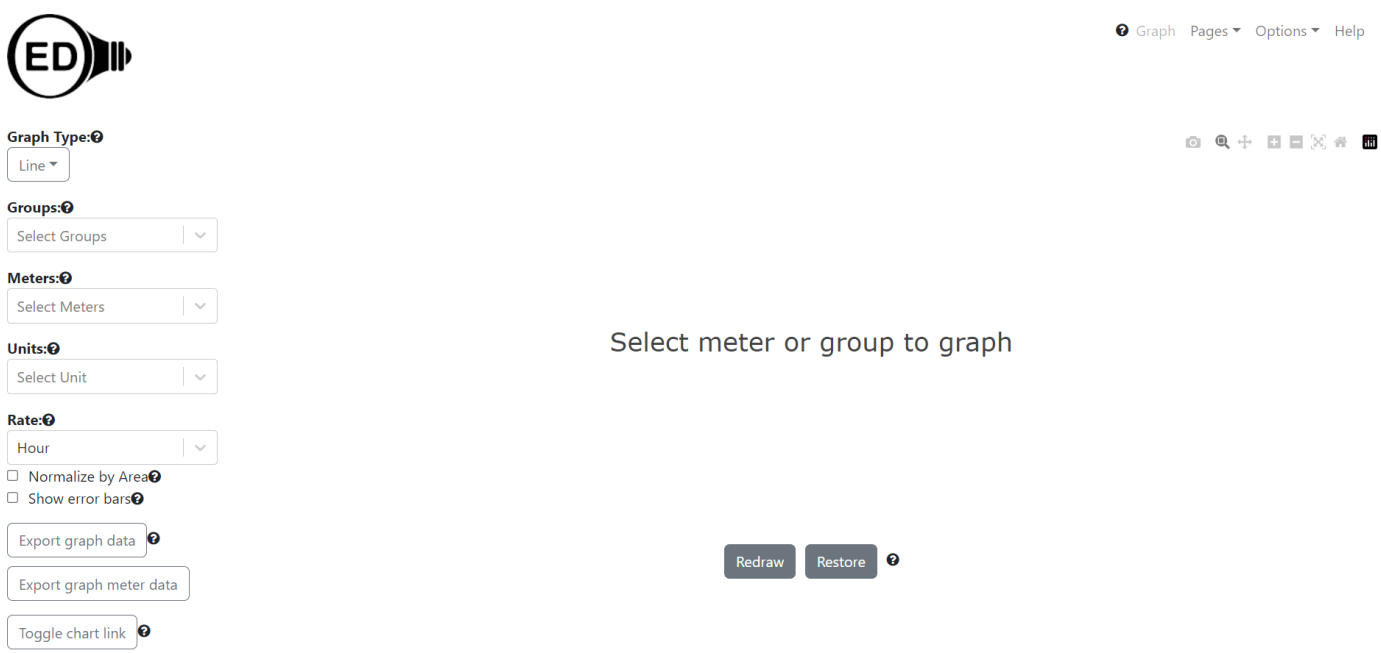
By Kartikeya Kumaria

"The primary goal of TDD is to make the code clearer, simpler, and bug-free."

- *Kent Beck*

Introduction

Approaching the end of 2023, I became a project contributor within a team of three people for the Open Energy Dashboard (OED), a web-based application that displays electrical energy usage information in a web browser. This project is vital as it extends to a diverse audience of organizations that have different approaches to collecting energy data, in which they can utilize for their own uses. When launching the OED, the user is greeted with the following dashboard in which they can integrate their data.

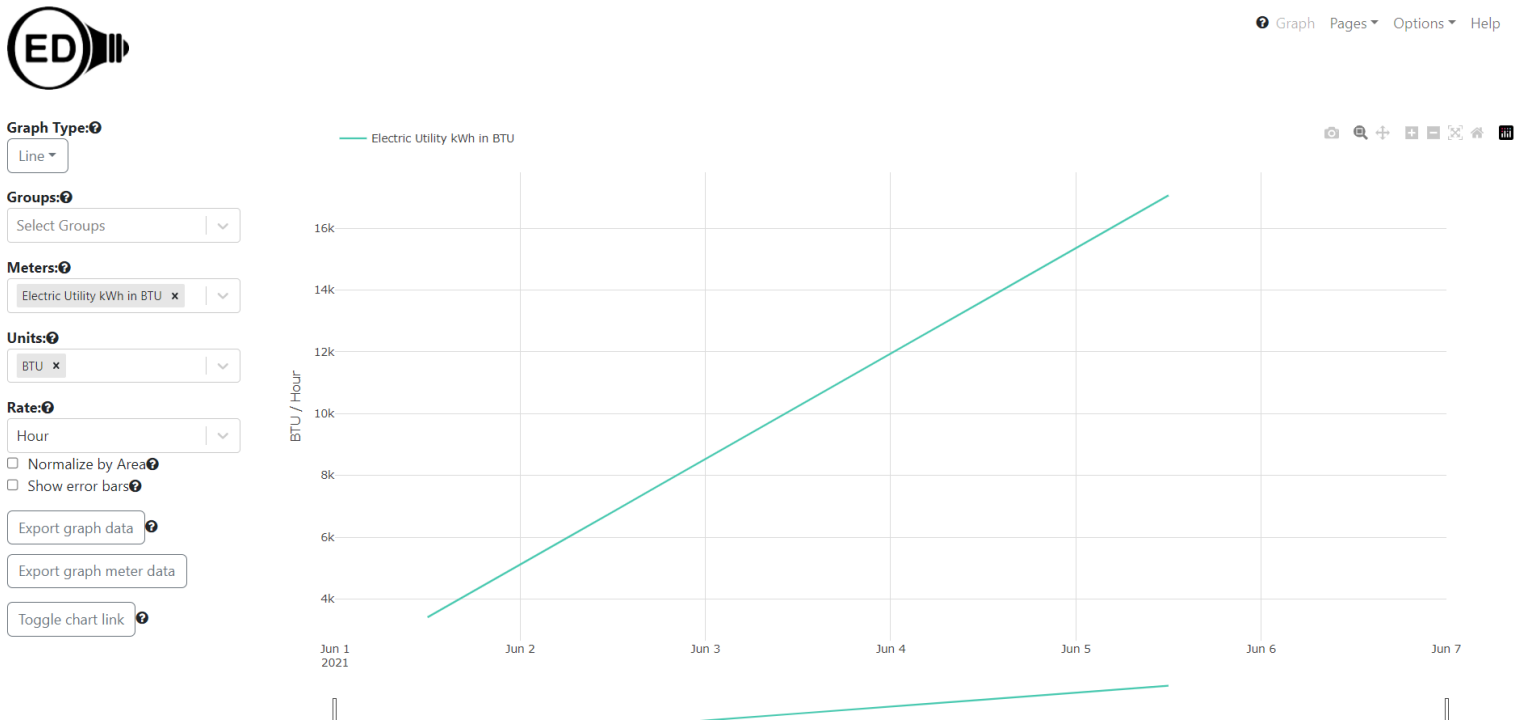


This application lets the user provide data to OED using an information providing hardware device called a meter. An example of this would be if an organization wants to collect information about a neighborhood's water usage, they would use a meter to collect the data and

write it in a .csv file, which would then get sent to OED in order to interpret the data. In order for meters to be used, the following terminal command must be run in order for the project maintainer to install data in their codebase.

```
# npm run testData
```

This enables developers to work with the application and to be able to interact with the front-end. Here is an example of a line graph that can be displayed.



Implementing The Test

The goal of this application is to provide general-wide use for all types of energy data, such as weight, volume, temperature, power, and more. There is, however, a consequence to providing this functionality for the end user as it increases the code complexity and the overall scale of the project. Notably, OED provides succinct documentation to allow outside developers facilitating the immense amount of test cases provided to improve the overall quality of the web application. I first started with implementing test B10 as provided in the documentation (fixing issue #962), which holds voltage data on Electrical Utility. For context, since OED is a node.js application, it uses the mocha.js javascript framework that runs on node.js for testing the reading bar meter quantity readings, in which my team worked on one of those readings. The

mocha test we were working on, test B10, essentially converts the kWh (kilowatt per hour) units of electrical utility to BTU units (British Thermal Units) using many of the units and conversions specified in the OED documentation. To begin, I needed to know what the project guidelines were for development to get started so I went to the OED developer pages and clicked the “Getting Started” link.

OED Website OED Help Developer Home Getting Started Working with OED ▼ Code Info & Tech resources ▼ Maintain ▼ Developer Details

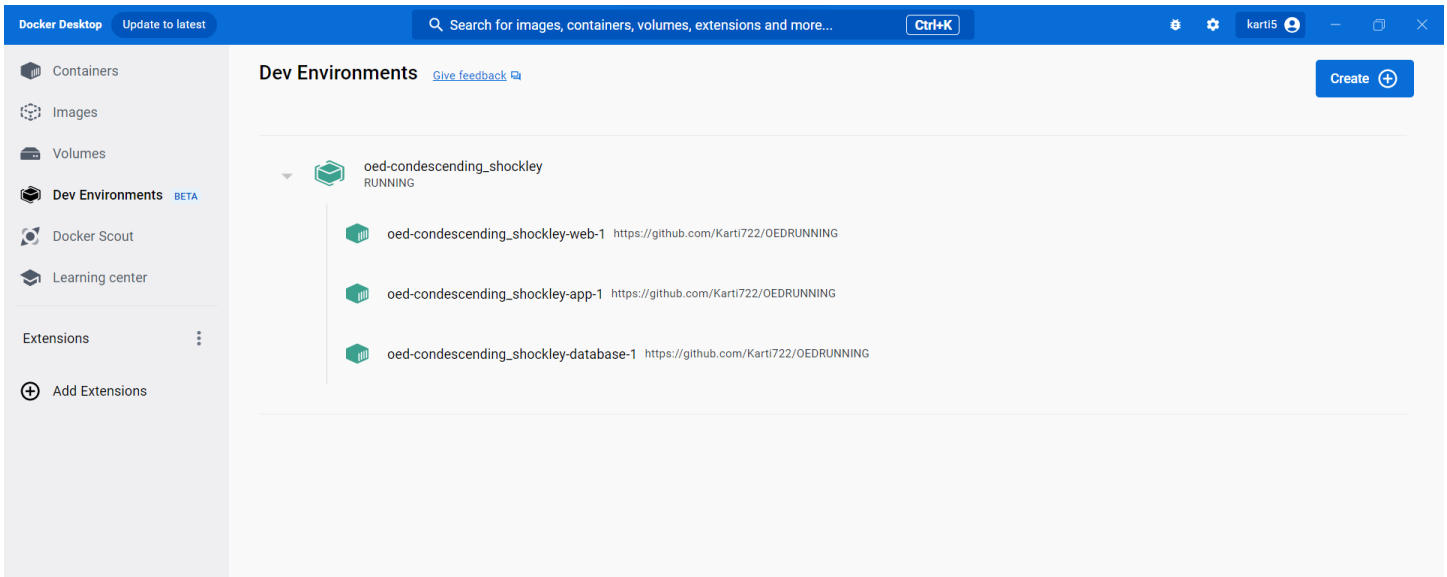
Getting started as an OED code developer

The following steps should get you ready to contribute to OED in a fairly short period of time. Please know that we are here to help you so use the link at the bottom of the page to email/message us with any issues.

Prerequisites

1. First, we strongly encourage you to contact the OED project as a potential developer. You can join our Discord channel with this Discord invite link or email us. That way we know about you and can help you select tasks that will be interesting to you and also benefit the project.
2. Before you can submit work to OED, you need to sign the Contributor's License Agreement.
3. The OED project resides on GitHub so you need a login to contribute to the project.
4. If you are using Windows 10+ then you will probably want to use the Windows Subsystem for Linux (WSL) that allows Docker to run on Windows. Please see this page for the steps you need to follow these extra steps. Installing WSL before Docker is helpful with the setup. **Please be sure to see the information on the WSL page (link earlier in this bullet) about using the Linux filesystem for OED and running Visual Studio Code under WSL if you are using Windows. You need to read this even if you already had WSL and/or Visual Studio Code.**
5. OED uses the Docker container system to run the software so you need to download and install Docker. (You need both Docker and docker compose but you generally get those together during the install.) There are paid subscriptions but your non-commercial work on the OSS OED project does not require a paid subscription. This has the advantage that our configuration files for Docker can get any needed software and also means we do not modify your normal system since the setup is in a separate container that does not impact other programs on your computer. Note that Docker should run on all OSes (Linux, Mac OS X, Windows - see above for information on using WSL on Windows) so you should be able to use it.
6. You are welcome to use any IDE for code development. Having said that, most OED developers use Visual Studio Code (VSC). The OED repository contains the .devcontainer folder that allows VSC to automatically launch and set up the needed Docker containers. There are a number of advantages to this system including that it puts in the extensions that OED thinks are valuable, gives a guaranteed environment to work with OED and avoids you having to separately install OED. Thus, you are encouraged to install VSC (if you don't already have it. The directions for working outside VSC follow the ones for VSC.
   - Once you have VSC installed, you need to add one extension to it so you can work with the OED container. Click on the Extensions manager icon on the far left of the VSC window (), search for "Remote Development" and install it. You will get several Extensions included with this one.
7. OED uses the fork model for development. (There are many descriptions of this process on the web but GitHub ones include a general one and one with more operational details.) This means you need to go to the main OED GitHub page (after login to GitHub), fork the project onto your own GitHub account and then clone the forked copy onto you own development machine to work on the code base. You can place the clone anywhere you want on your machine but please note what is stated on the WSL page linked above about where it should go on WSL. Note the clone will place all the files inside a new directory named OED unless you put a "." at the end of the `git clone` command. This model has the advantage that you can commit and push to your account without needing write permission on the main OED project repository. There are separate directions of the steps for creating a pull request to integrate your code into OED.

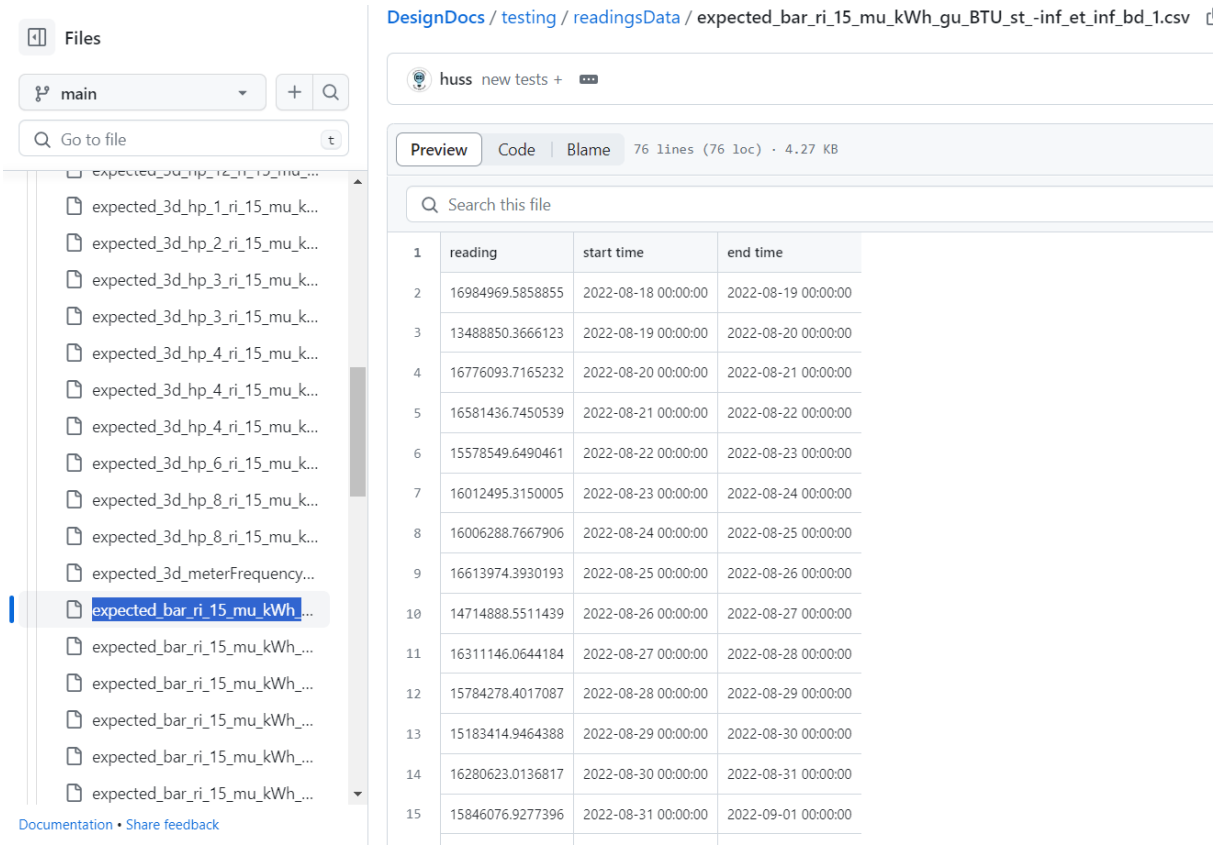
It was here where my team and I learned that we needed to fork the OED repo to my GitHub and add my team members as contributors, install Docker desktop, and install the VS Code Docker extension since I already had VS Code installed. From there, I went to Docker desktop and created a dev environment using the link for our team’s forked OED repo.



Then I launched the Docker repo in VS Code where I reopened it in a container using the installed Docker extension. From there, I went to the *src/server/test/web/readingsBarMeterQuantity.js* file (a javascript file that contained the scripts for running the bar meter quantity tests) file to include the rough draft code for test B10 as shown below.

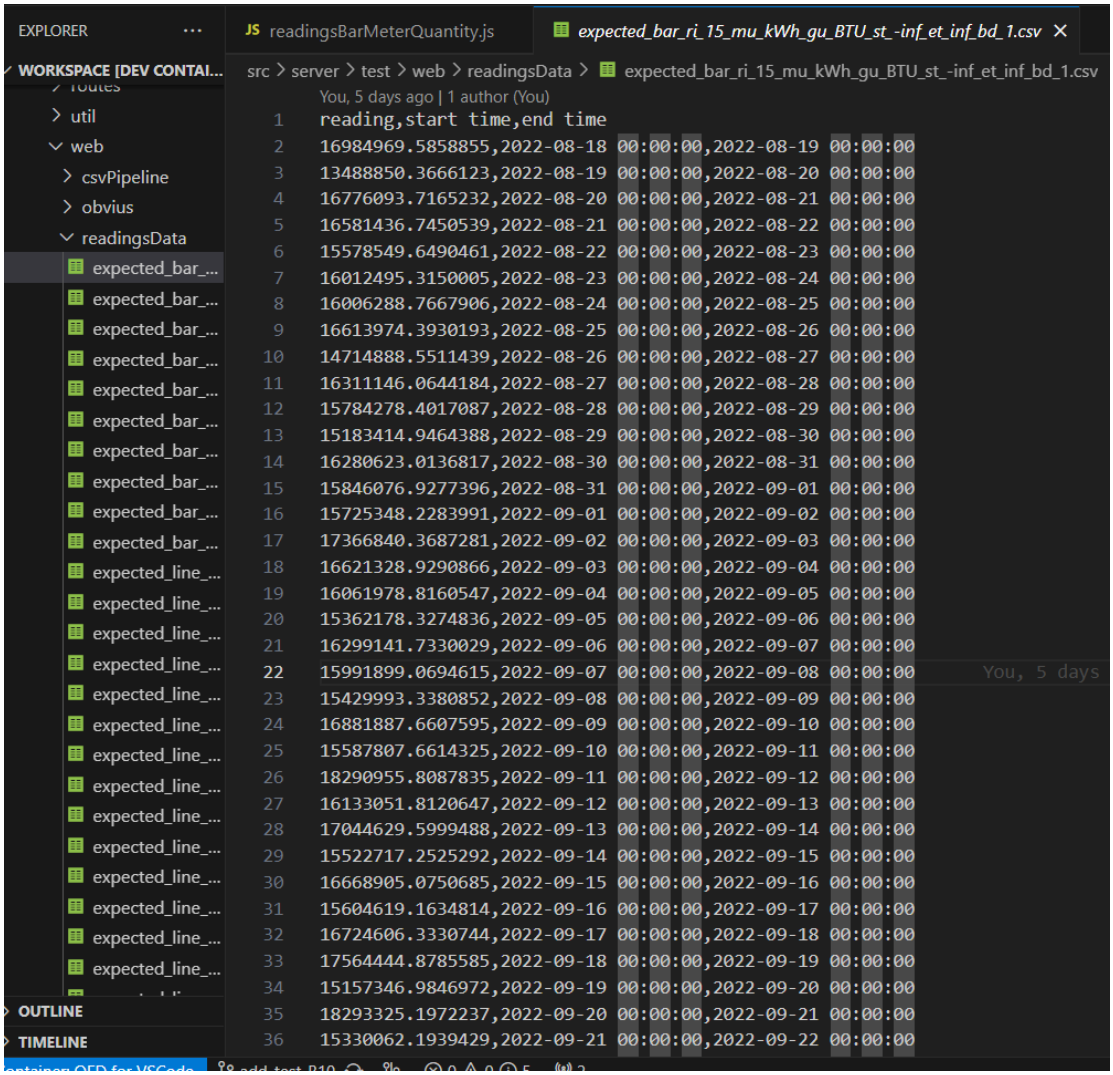
```
// Add B10 here
mocha.it('B10: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as BTU', async () => {
    // Load the data into the database

    await prepareTest(unitData, conversionData, meterData);
    // Get the unit ID since the DB could use any value.
    const unitId = await getUnitId('BTU');
    // Load the expected response data from the corresponding csv file
    const expected = await parseExpectedCsv('src/server/test/web/readingsData/expected_bar_ri_15_mu_kWh_gu_BTU_st_-inf_et_inf_bd_1.csv');
    // Create a request to the API for unbounded reading times and save the response
    const res = await chai.request(app).get(`/api/unitReadings/bar/meters/${METER_ID}`)
        .query({
            timeInterval: ETERNITY.toString(),
            barWidthDays: 1,
            graphicUnitId: unitId
        });
    // Check that the API reading is equal to what it is expected to equal
    expectReadingToEqualExpected(res, expected);
});
```

I quickly realized that the electrical utility readings for test B10 (contained in a csv file that was to be parsed for this test) didn't exist. So I went to the DesignDocs and found the csv file with all the readings.



1	reading	start time	end time
2	16984969.5858855	2022-08-18 00:00:00	2022-08-19 00:00:00
3	13488850.3666123	2022-08-19 00:00:00	2022-08-20 00:00:00
4	16776093.7165232	2022-08-20 00:00:00	2022-08-21 00:00:00
5	16581436.7450539	2022-08-21 00:00:00	2022-08-22 00:00:00
6	15578549.6490461	2022-08-22 00:00:00	2022-08-23 00:00:00
7	16012495.3150005	2022-08-23 00:00:00	2022-08-24 00:00:00
8	16006288.7667906	2022-08-24 00:00:00	2022-08-25 00:00:00
9	16613974.3930193	2022-08-25 00:00:00	2022-08-26 00:00:00
10	14714888.5511439	2022-08-26 00:00:00	2022-08-27 00:00:00
11	16311146.0644184	2022-08-27 00:00:00	2022-08-28 00:00:00
12	15784278.4017087	2022-08-28 00:00:00	2022-08-29 00:00:00
13	15183414.9464388	2022-08-29 00:00:00	2022-08-30 00:00:00
14	16280623.0136817	2022-08-30 00:00:00	2022-08-31 00:00:00
15	15846076.9277396	2022-08-31 00:00:00	2022-09-01 00:00:00

I placed the entire csv file into the *src/server/test/web/readingsData* directory, alongside the other csv files, and even matched the csv syntax accordingly.



I ran the test and found that an error occurred because I didn't yet implement the units/conversions appropriate for test B10 as specified in the DesignDocs.

u1, u2, u3, u16, c1, c2, c3

These units and conversions were all defined in the documentation so I implemented those definitions into the readingBarMeterQuantity.js file under the mocha test B10 code. Units u1 and u2 were already included in the code through the */workspace/src/server/util/readingsUtils.js* file

```
readingsUtils.js
src > server > util > readingsUtils.js > unitDatakWh
// These are the 2D arrays for units, conversions to feed into the database
// For kWh units.
const unitDatakWh = [        You, last week via PR #1087 • Removed excess units and conversions from reading
    {
        name: 'kWh',
        identifier: '',
        unitRepresent: Unit.unitRepresentType.QUANTITY,
        secInRate: 3600,
        typeOfUnit: Unit.unitType.UNIT,
        suffix: '',
        displayable: Unit.displayableType.ALL,
        preferredDisplay: true,
        note: 'OED created standard unit'
    },
    {
        name: 'Electric_Utility',
        identifier: '',
        unitRepresent: Unit.unitRepresentType.QUANTITY,
        secInRate: 3600,
        typeOfUnit: Unit.unitType.METER,
        suffix: '',
        displayable: Unit.displayableType.NONE,
        preferredDisplay: false,
        note: 'special unit'
    }
];
const conversionDatakWh = [
    {
        sourceName: 'Electric_Utility',
        destinationName: 'kWh',
        bidirectional: false,
        slope: 1,
        intercept: 0,
        note: 'Electric_Utility → kWh'
    }
];
```

which was imported to the readingsBarMeterQuantity.js as a variable called unitDatakWh, from which we created a new const variable called unitData that would replicate unitDatakWh and add the remaining needed units (u3 and u16) for our test. Similar story for the conversions in which c1 was already included in readingsutil.js file and was imported to the readingsBarMeterQuantity.js as conversionDataKwh and I created a new const variable called conversionData that would replicate conversionDataKwh and add the remaining conversions (c2 and c3) needed for the test to work successfully.

```
// Add B10 here
mocha.it('B10: 1 day bars for 15 minute reading intervals and quan
    // Load the data into the database
    const unitData = unitDatakWh.concat([
        {
            // u3
            name: 'MJ',
            identifier: 'megaJoules',
            unitRepresent: Unit.unitRepresentType.QUANTITY,
            secInRate: 3600,
            typeOfUnit: Unit.unitType.UNIT,
            suffix: '',
            displayable: Unit.displayableType.ALL,
            preferredDisplay: false,
            note: 'MJ'
        },
        {
            // u16
            name: 'BTU',
            identifier: '',
            unitRepresent: Unit.unitRepresentType.QUANTITY,
            secInRate: 3600,
            typeOfUnit: Unit.unitType.UNIT,
            suffix: '', displayable: Unit.displayableType.ALL,
            preferredDisplay: true,
            note: 'OED created standard unit'
        }
    ]);
```

```
const conversionData = conversionDatakWh.concat([
    {
        // c2
        sourceName: 'kWh',
        destinationName: 'MJ',
        bidirectional: true,
        slope: 3.6,
        intercept: 0,
        note: 'kWh → MJ'
    },
    {
        // c3
        sourceName: 'MJ',
        destinationName: 'BTU',
        bidirectional: true,
        slope: 947.8,
        intercept: 0,
        note: 'MJ → BTU'
    }
]);
```

And set up the meterData as well. We implemented the unitData, conversionData, and the meterData as parameters for the await prepareTest function below:

```
const meterData = [        You, 8 minutes ago • Uncommitted changes
    {
        name: 'Electric_Utility BTU',
        unit: 'Electric_Utility',
        displayable: true,
        gps: undefined,
        defaultGraphicUnit: "BTU",
        note: 'special meter',
        file: 'test/web/readingsData/readings_ri_15_days_75.csv',
        deleteFile: false,
        readingFrequency: '15 minutes',
        id: METER_ID
    }
];
await prepareTest(unitData, conversionData, meterData);
```

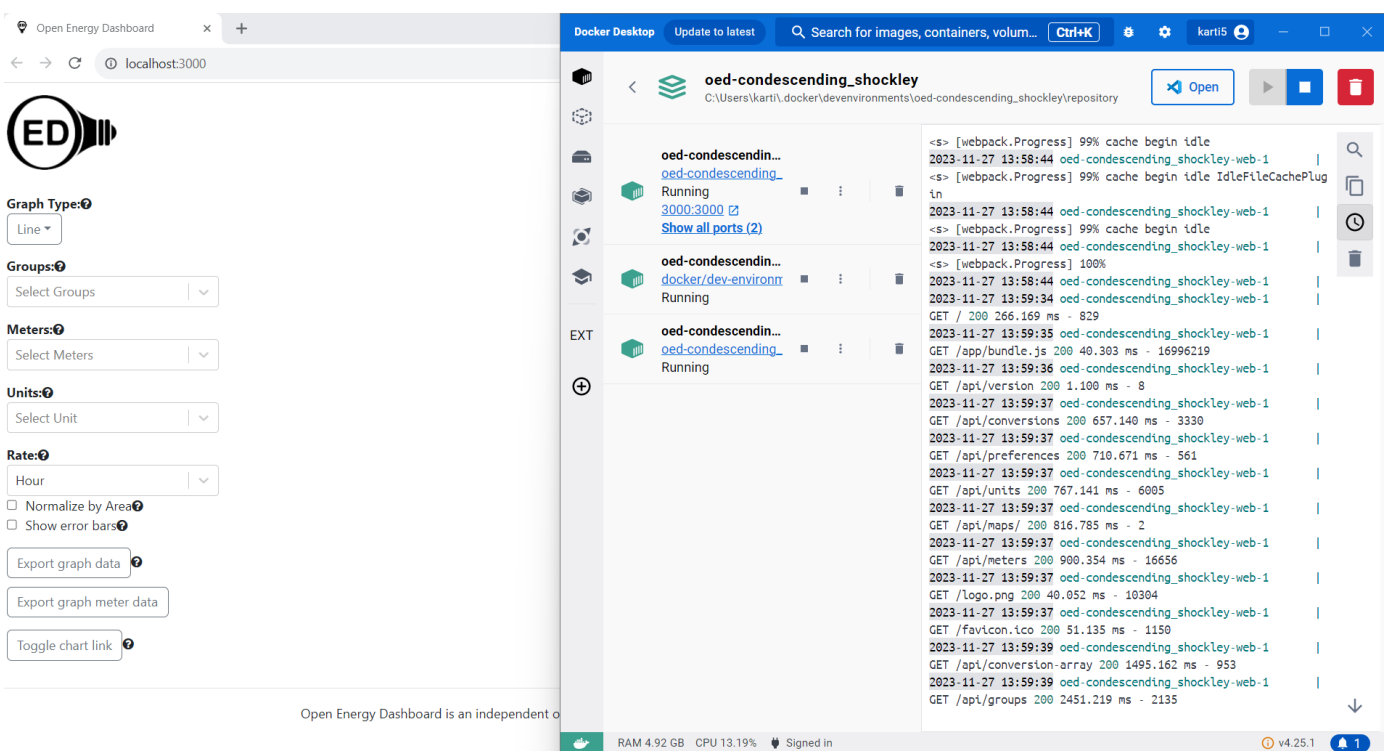
Managed to get Test B10 and other tests after syncing the forked repo with the main OED repo. I ran the following terminal command:

 # npm run testsome src/server/test/web/readingsBarMeterQuantity.js.

All the tests ran successfully.

```
readings API
  readings test, test if data returned by API is as expected
    for bar charts
      for quantity meters
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B1: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (4524ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B2: 7 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (3797ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B3: 28 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (3155ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B4: 13 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (3892ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B6: 76 day bars (no values) for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (3080ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B7: 13 day bars for 15 minute reading intervals and quantity units with reduced, partial days & kWh as kWh (3947ms)
          loading meter Electric Utility MJ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B8: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as MJ (3826ms)
          loading meter Electric_Utility BTU from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B10: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as BTU (3696ms)
          loading meter Electric_Utility metric ton of CO₂ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B13: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as metric ton of CO₂ & chained (3556ms)
          loading meter Electric_Utility pound of CO₂ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B14: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as lbs of CO2 & chained & reversed (3221ms)


10 passing (52s)
```

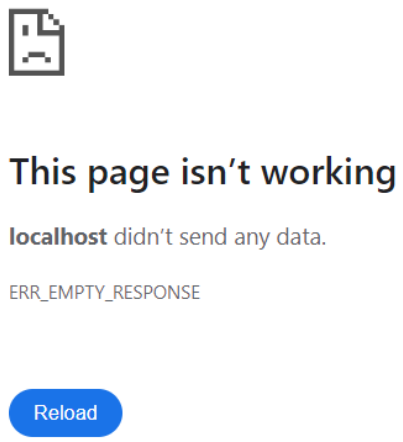
I committed the B10 test into my local git repository and synced it with the forked GitHub repo. I worked with other teammates to accomplish the final implementation and we even co-authored the last commit together, removing the compose-dev.yami file that wasn't needed. I also made sure the website still worked after all the code implementations we did as shown in the next page.



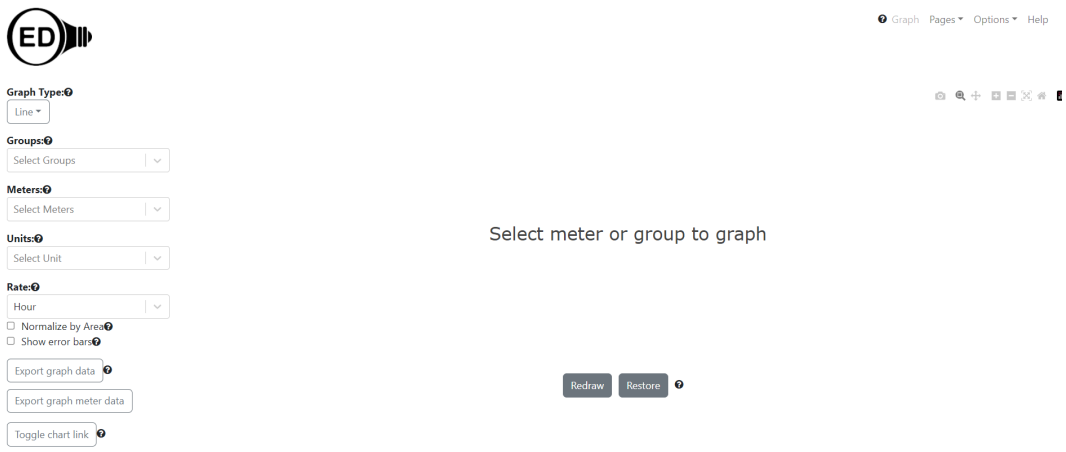
After ten commits made in our development branch (add-test-B10) and all changes pushed to our forked synced GitHub repo, my team created a pull request towards the main OED repo and our forked repo was successfully merged, thus ending our work.

Two Challenges

One challenge that my team faced when implementing test B10, was figuring out exactly how to run the codebase. When the OED repo was forked into my GitHub repository, we created a dev environment through Docker Desktop in order to run the dockerized application. When launching VS Code and running the website through the Docker extension, the website was hosted to http://localhost:3000/ where localhost generated an empty response.



My team and I eventually discovered that the VS Code Docker extension couldn't run the website because of specific compatibility issues with certain extensions and we didn't want to change them as it would potentially provoke merge conflicts during the pull request. We managed to get the website running only using Docker Desktop when running the dev environment, but not the repo.



Another unexpected issue was that when running the npm mocha tests through the VS Code container terminal (*npm run testsome src/server/test/web/readingsBarMeterQuantity.js)*, we got an error due to the VS Code container not having the mocha framework installed. I ran the command "*npm install mocha*" into the terminal and that fixed the issue as seen below.

```
readings API
  readings test, test if data returned by API is as expected
    for bar charts
      for quantity meters
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B1: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (2646ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B2: 7 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (2207ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B3: 28 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (2052ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B4: 13 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (2055ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B6: 76 day bars (no values) for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as kWh (2176ms)
          loading meter Electric Utility kWh from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B7: 13 day bars for 15 minute reading intervals and quantity units with reduced, partial days & kWh as kWh (2260ms)
          loading meter Electric Utility MJ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B8: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as MJ (2398ms)
          loading meter Electric_Utility BTU from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B10: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as BTU (2432ms)
          loading meter Electric_Utility metric ton of CO₂ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B13: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as metric ton of CO₂ & chained (2410ms)
          loading meter Electric_Utility pound of CO₂ from file test/web/readingsData/readings_ri_15_days_75.csv
            meter id set to  100
        ✓ B14: 1 day bars for 15 minute reading intervals and quantity units with +-inf start/end time & kWh as lbs of CO2 & chained & reversed (2768ms)
```

Those were two of the most glaring issues upon making a contribution to OED and I am happy that my team managed to overcome these two problems. We wouldn't have our pull request processed if we weren't persistently figuring out potential solutions to these issues.

Final Remarks

Test B10 was an essential component when it came to running bar meter quantity tests for electrical utility. This test was essential to tackle since OED was missing the BTU unit for

measuring electrical utility readings. The more units we have for testing the OED application, the more flexible the electric utility readings become which could be useful for users testing the data that may not use the same unit (one organization using kWh while another uses BTU). By adding the BTU unit, there is more flexibility when testing data.

Overall, we were satisfied with our contribution to the project and delivered a successful pull request (add-test-B10). This allowed our forked GitHub repository to merge with the OED main branch, thus successfully ending our task. We hope to deliver the same results with other projects in the future, allowing more secure, scalable, and powerful applications. Thank you for taking your valuable time to read this learning experience and hopefully, you can make some open source contributions as well! Here is our team's pull request linked below as a reference for our OED contribution.

Pull Request Link → **https://github.com/OpenEnergyDashboard/OED/pull/1087**